

Variáveis Latentes Discretas

K-Means, Misturas de Bernoulli e Gaussian Mixture Models

BLOCO	APRESENTADOR	TEMA	SLIDES	PALAVRAS	DURAÇÃO
Bloco 1	Henrique Matheus Mendonça de Miranda	Fundamentos e variáveis latentes	15	1991	14:45
Bloco 2	Luiany Gonçalves Carvalho	K-Means clustering	19	1962	14:32
Bloco 3	Antonio Carlos de Barcelos Fernandes	Misturas de Bernoulli e EM	18	1968	14:35
Bloco 4	Bianca Fernandes Visco	GMM, demonstração e comparação	15	1954	14:28
Total			67	7875	58:20

Cada entrada abaixo é o texto falado do slide correspondente, escrito em linguagem coloquial de apresentação. As durações são estimadas a 135 palavras por minuto, ritmo confortável de fala em português para conteúdo técnico — quem fala mais devagar deve somar cerca de 10%. Os quatro blocos ficam entre 12 e 15 minutos cada.

Fundamentos e variáveis latentes

Henrique Matheus Mendonça de Miranda

Slides 1-15 · 15 slides · 1991 palavras · $\approx$ 14:45 de fala

SLIDE 01 NÚCLEO

145 palavras · $\approx$ 1:04

Variáveis Latentes Discretas, K-Means e Modelos de Mistura

Bom dia a todos, professores, membros da banca e colegas. Meu nome é Henrique Miranda e, junto com a Luiany, o Antonio e a Bianca, eu vou apresentar nosso trabalho sobre variáveis latentes discretas, K-Means, misturas de Bernoulli e Gaussian Mixture Models. A ideia central que atravessa a apresentação inteira é esta: esses três métodos, que normalmente são ensinados em capítulos separados, são na verdade o mesmo modelo com hipóteses diferentes.

Nosso material tem setenta e três slides, mas a rota que vamos apresentar hoje são os vinte e oito slides de núcleo; o resto são laboratórios interativos e aprofundamentos que ficam pra consulta. A divisão é a seguinte: eu abro com os fundamentos e as variáveis latentes, a Luiany apresenta o K-Means, o Antonio as misturas de Bernoulli e o algoritmo EM, e a Bianca fecha com os modelos gaussianos e a comparação final.

SLIDE 02 NÚCLEO

127 palavras · $\approx$ 0:56

Sem rótulos, o objetivo deixa de ser prever e passa a ser descobrir

Vamos começar pela diferença que define tudo. No aprendizado supervisionado, o conjunto de dados vem em pares: uma entrada x e um rótulo y . Existe um gabarito externo, e o modelo ajusta suas fronteiras de decisão tentando errar o mínimo possível contra esse gabarito. No não supervisionado, esse gabarito simplesmente não existe.

A gente só tem as medições brutas. E isso muda a pergunta: não é mais 'qual é o rótulo desta amostra', e sim 'que estrutura existe aqui dentro'. Olhem os dois gráficos: são exatamente os mesmos duzentos e setenta pontos. À esquerda, as cores vêm dos rótulos verdadeiros.

À direita, elas não existem — só os contornos tracejados marcam os fatores geradores que a gente precisa inferir. O objetivo passa a ser descobrir, não prever.

Os três grandes objetivos do Aprendizado Não Supervisionado

E descobrir o quê, exatamente? A literatura organiza isso em três objetivos. O primeiro é agrupamento: particionar as observações em subpopulações homogêneas e disjuntas — é segmentação de clientes, taxonomia de espécies, descoberta de subtipos celulares. O segundo é estimação de densidade: modelar a função de probabilidade que rege os dados, quantificando onde eles são densos e onde são raros — isso é o que sustenta detecção de anomalia e de fraude, e a Bianca vai mostrar isso na prática no final.

O terceiro é aprender representações latentes: descrever cada amostra por poucos fatores interpretáveis, em vez das centenas de medidas originais. O bonito é que os modelos de mistura que a gente vai ver entregam os três de uma vez só — é o mesmo objeto matemático resolvendo as três tarefas.

Variável observada x , variável latente z : o que se mede e o que se infere

Aqui está o conceito que dá nome ao trabalho. A gente separa o mundo em duas variáveis. A observada, x , é o que o instrumento mediu: os pixels de uma foto, os sinais vitais de um paciente, as compras de um cliente. A latente, z , é o fator causal que a gente não mediu: o dígito que a pessoa pretendia escrever, a patologia de base, o perfil de consumo.

Ela existe, ela gerou o dado, mas ela não aparece na tabela. E a ponte formal entre as duas é a marginalização: a probabilidade de observar x é a soma, sobre todos os estados possíveis de z , do prior de z vezes a densidade condicional de x dado z .

Olhem o diagrama: o nó cinza é o único observado, o branco permanece oculto, e a caixa retangular indica que isso se repete para as N amostras.

Variáveis latentes no mundo real: 4 casos concretos

Pra isso não ficar abstrato, quatro casos concretos. Em visão computacional, o observado é uma grade de sessenta e quatro ou setecentos e oitenta e quatro pixels em preto e branco; o latente é o dígito que o escritor pretendia. Em medicina, o observado é um painel de cinquenta sintomas e biomarcadores; o latente é o subtipo da doença ou a mutação causadora.

Em processamento de texto, o observado é o vetor binário de presença de palavras; o latente é o tema editorial do artigo. E em finanças, o observado é valor, horário, geolocalização e velocidade da compra; o latente é o regime da transação — legítima ou fraudulenta.

Reparem no padrão: em todos, o latente é justamente aquilo que a gente gostaria de saber e não consegue medir direto.

O Modelo Gráfico Probabilístico e as independências condicionais

Este slide formaliza a estrutura com um modelo gráfico: é o mesmo desenho do slide quatro, agora com a placa em destaque. A placa é a caixa tracejada com o N no canto inferior direito, e ela quer dizer: tudo que está dentro se repete N vezes, uma cópia por amostra; o que está fora, π e θ , é global.

E a topologia do grafo não é decoração: ela define exatamente como a distribuição conjunta se fatora. A conjunta de todos os X e todos os Z é o produto, sobre as N amostras, do prior de z vezes a condicional de x dado z .

Daí saem duas consequências. A primeira é a independência amostral, que é o que permite escrever a verossimilhança como um produtório. A segunda é o acoplamento observado: ao marginalizar z , as observações deixam de ser independentes, porque passam a compartilhar π e θ . Essa tensão é exatamente o que vai tornar a otimização difícil e motivar o algoritmo EM lá no Bloco 3.

Codificação 1-de-K: um vetor binário para nomear o componente

Agora um detalhe de notação que parece burocrático e não é. A latente é categórica: ela diz de qual dos K componentes a amostra veio. A gente poderia representar isso com um número de um a K , mas em vez disso usa um vetor binário de tamanho K com exatamente um elemento igual a um — é a codificação um-de- K , ou one-hot.

O prior de pertencimento é π_k , a probabilidade de o componente k estar ativo, com todos os π somando um. E aí vem o truque: escrevendo o prior como o produtório de π_k elevado a z_k , como todos os z_k são zero exceto um, o produto colapsa automaticamente no π do componente ativo.

Isso parece um detalhe estético, mas é o que vai deixar o logaritmo tratável quando a gente derivar as equações do EM.

Laboratório: a mecânica 1-de-K e o colapso do produto

Esse laboratório é só pra fixar a mecânica do slide anterior, porque na primeira vez que a gente vê aquilo parece mágica. Cliquem em qualquer um dos quatro componentes: o vetor z muda, ficando um no componente escolhido e zero em todos os outros. E embaixo vocês veem o produto sendo montado termo a termo.

Qualquer número elevado a zero dá um, então todos os fatores inativos viram um e somem do produto por multiplicação. Sobra um único fator: o π do componente que está ligado. É só isso que a fórmula faz. A notação um-de-K é basicamente um interruptor algébrico que liga e desliga termos dentro de um produto, e é por isso que ela aparece em toda a literatura de modelos de mistura.

Três modelos, uma só estrutura: a hipótese sobre $p(x|z)$

Este é o slide que amarra a apresentação inteira, então eu peço atenção especial aqui. Existe um tronco comum: a probabilidade de x é a soma ponderada, sobre os K componentes, de π_k vezes a densidade condicional daquele componente. Essa fórmula não muda. O que muda de um método pro outro é uma única escolha: qual densidade condicional eu coloco ali dentro.

Se eu assumo variância desprezível e distância euclidiana, cai no K-Means, com atribuição rígida. Se o espaço é binário, eu coloco uma Bernoulli, e tenho as misturas de Bernoulli. Se o espaço é contínuo, eu coloco uma gaussiana, e tenho o GMM. À direita, os dois painéis mostram a mesma fronteira em duas linguagens: à esquerda o plano só admite duas cores chapadas; à direita a cor varia continuamente.

A árvore genealógica completa dos Modelos de Mistura

Aqui a mesma ideia como árvore genealógica, pra registrar que isso não para nos três casos de hoje. O tronco é sempre o mesmo, a latente um-de-K. Os ramos diferem só no domínio e na condicional escolhida. K-Means vive no espaço real contínuo, com uma Dirac ou o limite gaussiano quando a covariância vai a zero, atribuição rígida e custo de distorção.

Bernoulli vive no hipercubo binário, com atribuição suave e custo de log-verossimilhança. GMM vive no contínuo com elipsóides. E o quarto ramo mostra que qualquer distribuição da família exponencial serve: Poisson pra contagens, exponencial pra tempos entre eventos. A latente, o Passo E por Bayes e o Passo M por máxima verossimilhança ponderada continuam idênticos em todos eles.

Responsabilidade γ_{nk} : o posterior que Bayes devolve

E chegamos ao conceito central do meu bloco: a responsabilidade. A pergunta é: dado que eu observei a amostra x_n , qual a probabilidade de ela ter vindo do componente k ? A resposta é Bayes puro: no numerador, o prior π_k vezes a verossimilhança daquele componente; no denominador, a soma dos mesmos termos sobre todos os K componentes, que é a evidência.

Esse número, que a gente chama γ_{nk} , é o Passo E de todo algoritmo EM que vocês vão ver nos próximos blocos — é literalmente a mesma fórmula, muda só a densidade que entra dentro dela. No gráfico, arrastem o controle e vejam: quando o ponto está bem no meio de um componente, a responsabilidade dele vai perto de um; quando cai na fronteira, as duas barras ficam próximas de cinquenta por cento.

Laboratório: cálculo de γ_{nk} passo a passo na mão

Este laboratório é a mesma conta do slide anterior, mas com os números todos abertos na tela, porque na hora da prova é assim que a gente confere. A tabela tem uma linha por componente. Primeira coluna, o prior. Segunda, a verossimilhança daquele componente na posição da amostra.

Terceira, o produto dos dois, que é o numerador. E a quarta é a responsabilidade, que é o numerador dividido pela soma dos numeradores. Movam o controle da posição da amostra e acompanhem: quando ela caminha pra esquerda, a responsabilidade do componente um sobe; quando caminha pra direita, sobe a do dois.

E, aconteça o que acontecer, a soma das duas dá exatamente um ponto zero zero zero, porque a normalização garante isso por construção.

Código Python: calculando responsabilidades γ_{nk} do zero

Pra quem quiser reproduzir, aqui está a implementação em NumPy puro, sem biblioteca de machine learning. São três etapas, marcadas nos comentários. Primeiro, para cada componente, a gente avalia a densidade gaussiana multivariada — inverte a covariância, calcula o determinante e a distância de Mahalanobis ao quadrado.

Depois multiplica pelo prior, o que o NumPy faz por broadcasting numa linha só. E por fim divide pela evidência, que é a soma sobre os componentes com keepdims igual a True, pra manter a forma da matriz. Ao lado está a saída real da execução: reparem na segunda amostra, que caiu na fronteira e deu quarenta por cento contra cinquenta e nove.

E na última linha, a soma de cada linha dando exatamente um. Isso não é sorte, é a normalização.

Soft vs Hard Assignment: incerteza probabilística vs decisão binária

E aí chegamos na diferença conceitual que vai separar o bloco da Luiany dos blocos do Antonio e da Bianca. Na atribuição suave, cada ponto distribui seu pertencimento entre todos os clusters através de um vetor de probabilidades que soma um. A vantagem é preservar a incerteza real nas regiões de sobreposição, e dar gradientes contínuos pra otimização.

Na atribuição rígida, o ponto pertence integralmente a um único cluster: o vencedor leva um, os demais levam zero. A vantagem é a simplicidade e a velocidade. A desvantagem é que a incerteza na fronteira é descartada — um ponto que estava cinquenta e um a quarenta e nove é tratado exatamente como um ponto que estava noventa e nove a um.

Essa informação some, e não volta.

Síntese do Bloco 1 — e a pergunta que abre o K-Means

Fechando o meu bloco, três conquistas. Primeira: o aprendizado não supervisionado busca densidades e regularidades latentes, não rótulos. Segunda: a codificação um-de-K expressa a incerteza de pertencimento numa notação que o logaritmo consegue tratar. Terceira: o Teorema de Bayes converte prior vezes verossimilhança em responsabilidades, e essa é a engrenagem que vai reaparecer em todos os métodos daqui pra frente.

E fica a pergunta que abre o próximo bloco: e se a gente eliminar as probabilidades por completo, e resolver o agrupamento de forma puramente geométrica e determinística no espaço euclidiano? O que a gente ganha em velocidade e o que a gente perde? Pra responder isso, eu passo a palavra pra Luiany, que vai apresentar o K-Means.

Obrigado.

K-Means clustering

Luiany Gonçalves Carvalho

Slides 16–34 · 19 slides · 1962 palavras · $\approx$ 14:32 de fala

SLIDE 16 NÚCLEO

120 palavras · $\approx$ 0:53

K-Means: retalhar o espaço em K células de Voronoi

Obrigada, Henrique. Meu nome é Luiany Carvalho e eu vou conduzir o Bloco 2, que é onde a gente coloca a mão na massa com o primeiro algoritmo de agrupamento: o K-Means. Ele é a versão mais crua da ideia de variável latente. Eu escolho um número K de grupos, represento cada grupo por um único ponto, o centróide, e todo mundo pertence ao centróide mais perto.

Olhem a figura: quando eu fixo os centróides, o espaço se reparte sozinho em células de Voronoi, com fronteiras equidistantes entre centros vizinhos. E guardem uma palavra que vai voltar o tempo todo: a atribuição aqui é rígida. O r_{nk} vale zero ou um. É justamente isso que o Bloco 4 vai relaxar.

SLIDE 17 NÚCLEO

109 palavras · $\approx$ 0:48

A função de distorção J: o que exatamente está sendo minimizado

Pra escolher centróides eu preciso de um critério que diga que uma escolha é melhor que a outra: a função de distorção, também chamada de inércia. A fórmula assusta, mas leiam em português: pra cada ponto pego a distância até o centróide do grupo dele, elevo ao quadrado e somo tudo.

Então J é literalmente a soma dos tracejados do desenho — quanto menor, mais grudados os pontos estão nos seus centros. E aqui aparece o problema: J depende de duas coisas ao mesmo tempo, de quem pertence a quem e de onde estão os centros.

Testar todas as combinações é NP-difícil, então a gente vai de estratégia iterativa.

Laboratório: calculando a distorção J ponto a ponto na mão

Antes do algoritmo, vamos calcular esse J na mão, porque quando a gente calcula uma vez fica muito mais concreto. Para cada ponto eu meço a distância até cada centro, elevo ao quadrado, escolho a menor, que é a que entra na conta, e vou acumulando.

E olhem: quando eu movo um centróide pro miolo do seu grupo, o total despenca. É essa a intuição que o algoritmo explora — toda decisão daqui pra frente só vale se derrubar esse número.

Algoritmo de Lloyd: descida alternada em duas coordenadas

O algoritmo de Lloyd resolve o impasse com um truque simples: como é difícil otimizar as duas coisas juntas, ele congela uma e otimiza a outra, alternando. Na atribuição eu fixo os centróides e pergunto pra cada ponto qual o centro mais perto; cada ponto decide sozinho.

Na atualização eu faço o contrário: congelo as atribuições e pergunto onde o centro deveria estar, e a resposta é a média do grupo. Duas garantias: cada passo, sozinho, nunca aumenta o J; e como o número de partições é finito, o algoritmo obrigatoriamente para.

Ele converge sempre — só que pra um mínimo local, não necessariamente pro melhor de todos.

A prova matemática: por que a média minimiza a soma dos quadrados?

Eu disse que o centro ótimo é a média; vamos provar isso. Fixando as atribuições, o custo do cluster depende só do μ dele. Abro o quadrado e sobram três termos: um que não depende de μ , um cruzado e um quadrático. Derivo, igualo a zero, e o termo constante some.

Sobra uma equação linear: a soma dos pontos menos N_k vezes μ igual a zero. Isolando, μ é a média aritmética. E a segunda derivada dá dois vezes N_k , sempre positivo, então é mínimo mesmo. A média não é bom senso, ela cai da matemática.

Laboratório: por que a média? A parábola de $J(\mu)$ e a derivada zero

Essa mesma prova, agora em desenho e em uma dimensão só. O eixo horizontal é onde eu coloco o centróide e o vertical é o custo. Conforme eu arrasto o centro, a curva desenha uma parábola com a boca pra cima — e tem que ser parábola, porque J é soma de termos quadráticos em μ .

Reparem que o fundo dela, onde a tangente fica horizontal, cai exatamente em cima da média dos pontos. Se eu empurro pra qualquer lado, a derivada deixa de ser zero e o custo sobe.

Laboratório: Lloyd meio-passo a meio-passo — Atribuir vs Atualizar

Agora vamos separar bem os dois meios-passos, porque é aqui que muita gente se confunde. Vou clicar e só atribuir: os centros ficam parados e são as cores que mudam. Clico de novo e só atualizo: as cores param e os centros se deslocam pro meio das suas nuvens.

Percebam o padrão — a cada meio-passo o J cai, ou no máximo empata, mas nunca sobe. É a descida alternada acontecendo na frente de vocês. E o movimento vai ficando menor até que um clique não muda mais nada: quando nenhum ponto troca de cor, convergiu.

Lloyd em execução: quatro iterações e a queda monótona de J

Aqui a mesma coisa numa escala maior: duzentos e setenta pontos, três grupos, com o algoritmo rodando de verdade dentro do navegador. À esquerda, a trajetória dos centróides, que sai de uma posição aleatória e caminha até o coração de cada nuvem. À direita, a curva do J .

Olhem o formato: despenca da primeira pra segunda iteração e depois vira quase uma reta. Na prática o K-Means resolve quase tudo nas duas ou três primeiras rodadas e o resto é ajuste fino. O custo de cada iteração é linear no número de pontos: por isso um algoritmo dos anos setenta segue rodando em conjuntos gigantes.

Escolher K: o cotovelo da inércia e o pico da silhueta

Até agora eu venho dizendo "escolho K" como se fosse trivial, e não é. Tem uma armadilha séria: o J sempre melhora quando o K aumenta. Se eu botar K igual ao número de pontos, cada ponto vira seu próprio centro e o J dá zero — inércia perfeita e informação nenhuma.

O primeiro critério é o cotovelo, à esquerda: procuro onde a curva para de despencar, porque dali pra frente eu pago um grupo a mais e ganho quase nada. O segundo é a silhueta, à direita, e essa tem um pico de verdade. Aqui os dois apontam pro mesmo K igual a quatro — quando eles concordam, a gente fica bem mais tranquila.

A matemática da Silhueta: coesão $a(i)$, separação $b(i)$ e interpretação

Vale abrir a silhueta, porque a fórmula é intuitiva. Para um ponto i , o a de i é a distância média dele até os colegas do mesmo cluster: é a coesão, o quanto ele está em casa. O b de i é a distância média até o cluster vizinho mais próximo, ou seja, o concorrente mais perigoso.

A silhueta é a diferença entre os dois dividida pelo maior deles, espremendo tudo entre menos um e mais um. Perto de mais um, ponto bem classificado. Perto de zero, ele está em cima da fronteira. E se der negativo é alarme: aquele ponto está mais perto do grupo vizinho do que do grupo dele.

Laboratório: ajustando K de 1 a 8 com inércia e silhueta ao vivo

Agora dá pra brincar com o K ao vivo. Vou arrastar de um até oito. Com K igual a um, o J é enorme, é a variância total dos dados. Puxando pra dois, três, quatro, a queda é bem visível. Passou do quatro, reparem: o J continua caindo, sim, mas devagarzinho, e é aí que a silhueta começa a desabar, porque eu comecei a rachar grupos que eram um só.

O cotovelo diz quando parar de ganhar; a silhueta diz quando começar a perder.

Inicialização e escala: dois modos silenciosos de estragar o resultado

Duas coisas estragam o K-Means silenciosamente, sem dar erro, entregando um resultado bonito e errado. A primeira é a inicialização. À esquerda, uma semente infeliz: dois centros nasceram dentro da mesma nuvem e brigam pra dividir aquele grupo, enquanto duas nuvens de verdade lá longe ficaram fundidas num centro só.

E ele convergiu: não tem nada quebrado, só caiu num mínimo local ruim. À direita, os mesmos dados semeados de outro jeito, e os grupos aparecem certinhos. A ideia do k-means++ é espalhar os centros iniciais de propósito: cada novo centro é sorteado com probabilidade proporcional ao quadrado da distância até o centro mais próximo que já existe.

A matemática do k-means++: por que a probabilidade $\propto D(x)^2$?

Deixa eu detalhar essa semente. O primeiro centro é sorteado uniformemente. Depois, pra cada ponto eu calculo D de x , a distância até o centro mais próximo já escolhido, e sorteo o próximo com probabilidade proporcional a D ao quadrado. Duas perguntas sempre aparecem. Por que sortear, e não pegar o ponto mais distante?

Porque o mais distante costuma ser um outlier, e o determinístico sentaria um centróide em cima de ruído. E por que ao quadrado? Porque o quadrado exagera a diferença: uma região duas vezes mais longe fica quatro vezes mais provável.

Laboratório: a roleta do k-means++ semeando centros com probabilidade $\propto D^2$

Aqui está essa roleta funcionando. Cada ponto do gráfico virou uma fatia da barra de probabilidade lá embaixo, e o tamanho da fatia é o D ao quadrado dele. Coloquei o primeiro centro aqui, e reparem: todos os pontos perto dele encolheram até quase sumir da barra, e as nuvens distantes ficaram com fatias enormes.

Vou sortear: caiu numa nuvem longe, como era esperado — não garantido, mas muito mais provável. Na terceira rodada, a única que ainda tem massa na roleta é justamente a nuvem que falta.

O perigo da escala: por que padronizar atributos é obrigatório

O segundo modo silencioso de errar é a escala, e esse é mais traiçoeiro porque não depende de sorte: ele erra sempre. A distância euclidiana soma os quadrados das diferenças de cada atributo. Imaginem uma tabela com salário anual e idade. Cinquenta mil reais ao quadrado dá dois vírgula cinco vezes dez elevado a nove; quarenta anos ao quadrado dá mil e seiscentos.

O termo da idade é um milhão de vezes menor — existe na fórmula, mas não influencia nada. Você achou que agrupava por perfil e agrupou por faixa salarial. A correção é padronizar com z-score, e ajustar o scaler só no treino, senão você vaza informação.

Código Python: K-Means do zero em 25 linhas de NumPy

Esse é o K-Means inteiro escrito do zero, e cabe em vinte e cinco linhas de NumPy. Destaco a linha do broadcasting: esse X com `newaxis` menos os centróides monta de uma vez só todas as N vezes K diferenças, sem nenhum laço em Python.

À direita está a saída real da execução. Gerei três nuvens gaussianas centradas em menos quatro menos dois, zero e três, e quatro menos um. Ele convergiu em quatro iterações e recuperou os três centros com erro abaixo de zero vírgula três. Os tamanhos deram sessenta, sessenta e um e cinquenta e nove: dois pontos de fronteira trocaram de grupo, o que é esperado quando as nuvens se encostam.

Laboratório: K-Means em imagens (quantização e compressão de cores)

Uma aplicação que fecha bem o bloco: quantização de cores. Numa imagem, cada pixel é um ponto em três dimensões — vermelho, verde e azul. Se eu rodo K-Means nesse espaço com K igual a dezesseis, acho as dezesseis cores que melhor representam a imagem inteira e troco cada pixel pela cor do seu centróide.

Com K bem baixo a imagem fica cartunizada, aparecem blocos chapados; aumentando o K ela volta ao normal, e por volta de trinta e duas cores o olho quase não distingue do original.

Onde o K-Means falha — e por que isso motiva os modelos de mistura

E agora a parte mais importante do meu bloco: onde esse algoritmo quebra. O K-Means tem uma hipótese geométrica escondida na própria definição: como só usa distância euclidiana até o centróide, ele acredita que todo grupo é uma bolinha redonda e que todas têm mais ou menos o mesmo tamanho.

Quando isso não vale, ele erra feio. À esquerda, faixas diagonais alongadas: as fronteiras de Voronoi são retas, então ele corta as faixas atravessado. À direita, densidades desiguais: ele fatia o grupo largo e funde os dois compactos, porque isso dá um J menor. E tem um terceiro caso: em vetores de zeros e uns, a distância euclidiana perde o sentido e a média vira um número quebrado que não representa objeto nenhum.

Síntese do Bloco 2 — e a transição para dados discretos

Fechando o Bloco 2. O que fica é: o Lloyd minimiza a inércia por descida alternada, com convergência garantida mas local; a escolha do K precisa de cotovelo e silhueta juntos; e a sementeira k-means++ com a padronização decidem se o resultado presta ou não.

E fica a dívida do slide anterior. As três falhas têm a mesma raiz: atribuição rígida e distância euclidiana, sem nenhuma noção de forma, de espalhamento, de incerteza. A saída é trocar distância por probabilidade: em vez de perguntar qual centro está mais perto, perguntar qual distribuição tem mais chance de ter gerado aquele ponto.

E é isso que o Antonio vai apresentar agora, com as misturas de Bernoulli e o algoritmo EM. Antonio, é com você.

Misturas de Bernoulli e EM

Antonio Carlos de Barcelos Fernandes

Slides 35–52 · 18 slides · 1968 palavras · $\approx$ 14:35 de fala

SLIDE 35 NÚCLEO

130 palavras · $\approx$ 0:58

Dados binários vivem num hipercubo — e ali a média euclidiana falha

Obrigado, Luiany. Eu sou o Antonio Fernandes e no Bloco 3 a gente ataca a dívida que ela deixou. Olhem o tipo de dado da tabela: cada coluna é um bit, zero ou um. Isso aparece em todo lugar: imagem preto e branco, onde o pixel acende ou não; texto, onde a palavra aparece ou não; ficha clínica, com o sintoma marcado sim ou não.

Esses dados não vivem num plano: vivem nos vértices de um hipercubo. E o K-Means quebra de um jeito concreto: a média de um monte de bits dá, por exemplo, zero vírgula quarenta e três. Esse número não é vértice nenhum do cubo: não é um documento possível, nem uma imagem possível.

O protótipo que o algoritmo devolve não existe no mundo dos meus dados.

SLIDE 36 NÚCLEO

118 palavras · $\approx$ 0:52

Bernoulli multivariada: D moedas independentes por componente

A saída é trocar distância por probabilidade. Se cada dimensão é um bit, a distribuição natural dela é a Bernoulli, uma moeda viciada. Então eu modelo cada grupo como D moedas independentes, uma por dimensão — é a fórmula do slide. O μ_{kd} é a probabilidade daquele bit acender dado que a amostra veio do grupo k.

Leiam o produto com calma: quando x_d é um, sobra o μ ; quando x_d é zero, sobra um menos μ . É um jeito compacto de multiplicar a probabilidade de cada bit ter dado o que deu. E reparem: o mesmo vetor μ determina de uma vez a média e a variância de cada bit — não existe parâmetro de dispersão separado.

A imagem de probabilidades: o que é exatamente o vetor μ_k ?

Vale insistir nessa mudança. No K-Means o μ era um ponto geométrico, um lugar no espaço; aqui ele é uma coleção de D probabilidades de ativação. Pensem no dígito zero numa grade oito por oito: os pixels do anel acendem quase sempre, μ perto de zero vírgula noventa e cinco; os do buraco no meio quase nunca acendem, μ perto de zero vírgula zero cinco.

Ou seja, o protótipo não é um dígito que alguém desenhou: é um retrato borrado que responde com que frequência cada pixel acende quando este grupo gera uma imagem.

Laboratório: pinte um dígito 8×8 e veja o reconhecimento ao vivo

Pra deixar palpável, aqui dá pra desenhar. Eu clico nos quadradinhos e monto um dígito na grade. Repare que, a cada bit que eu ligo, as barras de probabilidade ao lado se mexem na hora: o modelo está calculando, para cada protótipo, a chance de aquele desenho ter saído dele.

Vou desenhar um zero bem feito e o componente do zero dispara; apago um pedaço e a confiança cai, com um segundo componente disputando. É isso que a gente vai chamar de responsabilidade: não é resposta seca, é divisão de crédito.

O somatório preso dentro do logaritmo: por que não há solução fechada

Agora a dificuldade técnica do bloco. O modelo completo é uma mistura: somo, sobre os K grupos, o peso π_k vezes a Bernoulli daquele grupo. O caminho natural pra ajustar os parâmetros seria a máxima verossimilhança: escrever o log, derivar, igualar a zero. Só que olhem onde o somatório caiu — ele ficou preso dentro do logaritmo.

E logaritmo de soma não se abre: não existe identidade que separe $\ln$ de a mais b . Se os grupos fossem conhecidos, o log entraria no produto e tudo viraria conta trivial; como não são, a derivada dá um sistema acoplado, sem solução fechada. É esse impasse que motiva o EM.

Laboratório: por que o log atrapalha ($\ln(a+b) \neq \ln a + \ln b$)

Esse laboratório existe só pra fixar essa pedra no caminho. À esquerda eu tenho $\ln$ de a mais b ; à direita, $\ln a$ mais $\ln b$. Vou mexer nos valores: os dois números quase nunca batem, e a diferença entre eles nem é constante, muda conforme os valores.

Ou seja, não é um erro que eu corrija com uma constante. Por isso a máxima verossimilhança direta trava aqui, e por isso o EM não tenta abrir esse logaritmo: ele contorna, trocando a função difícil por uma auxiliar que a gente sabe otimizar.

O Algoritmo EM: uma estratégia elegante para dados incompletos

A ideia do EM é resolver um ovo e galinha. Se eu soubesse de qual grupo veio cada amostra, ajustar os parâmetros seria trivial: era só separar e tirar médias. E se eu soubesse os parâmetros, descobrir o grupo seria trivial: era só aplicar Bayes.

O problema é que eu não sei nenhum dos dois. Então o EM chuta os parâmetros, usa o chute pra calcular a distribuição sobre os grupos, e usa essa distribuição pra recalculer os parâmetros. E repete. É a mesma alternância do Lloyd, com uma diferença crucial: lá o ponto ia inteiro pra um grupo, aqui ele vai fracionado pra todos.

EM para Misturas de Bernoulli: duas equações fechadas, alternadas

Na mistura de Bernoulli, os dois passos ficam em fórmula fechada. No passo E eu calculo a responsabilidade γ_{nk} : o peso do grupo vezes a probabilidade que ele dá pra amostra, dividido pela soma de todos. É Bayes puro, é a posteriori que o Henrique apresentou no Bloco 1, e soma um em cada linha.

No passo M eu somo as responsabilidades e obtenho N_k , o tamanho efetivo do grupo — um número quebrado, tipo trinta e sete vírgula dois. O π_k é esse tamanho dividido por N , e o μ_k é a média das amostras ponderada pelas responsabilidades. É a mesma média do K-Means, só que cada amostra entra com o peso do quanto ela pertence.

Laboratório: EM passo a passo em 6 amostras binárias

Vamos rodar com seis amostras na unha. Aqui estão os seis vetores binários e dois componentes com parâmetros chutados. Passo E: para cada amostra eu calculo a probabilidade sob cada componente, multiplico pelo peso e normalizo. Olhem a tabela: essa amostra ficou noventa e três por cento no grupo um; essa outra ficou cinquenta e dois contra quarenta e oito, com o modelo honestamente em dúvida.

Passo M: os mus se deslocam na direção das amostras que mais os reivindicaram. Clico de novo e as dúvidas vão sumindo, com as responsabilidades cada vez mais perto de zero e um. E o log da verossimilhança lá embaixo sobe a cada rodada.

A prova da convergência: a decomposição $L(q, \theta) + KL(q \parallel p)$

E por que ele sobe sempre? Essa é a prova de convergência. Para qualquer distribuição q sobre a latente, o log da verossimilhança se decompõe em duas parcelas exatas: uma cota inferior, o ELBO, mais a divergência de Kullback-Leibler entre q e a posteriori verdadeira.

Como a KL nunca é negativa, o ELBO fica sempre por baixo. Agora vejam a mágica. No passo E, eu escolho q igual à posteriori exata: isso zera a KL, e a cota encosta na curva. No passo M, eu subo a cota mexendo nos parâmetros.

Como a cota subiu e a verossimilhança está sempre acima dela, ela também subiu. O EM nunca piora: no pior caso empata, e aí convergiu.

Laboratório: por que $\ln p(X)$ nunca cai — visualizando a cota ELBO

Aqui está o argumento em desenho. A curva de cima é o log da verossimilhança, que eu não sei otimizar direto; a de baixo é a cota. No passo E a cota é levantada até tocar a curva no parâmetro atual: as duas se beijam, a KL virou zero.

No passo M eu ando pro máximo da cota, que é fácil, e o ponto escorrega pra direita. E a garantia visual é essa: como a cota nunca ultrapassa a curva de cima e eu sempre subo a cota, a curva de cima nunca desce.

E uma consequência prática: se no seu código o log da verossimilhança cair, não é o EM, é bug.

Protótipos emergem do ruído: EM sobre dígitos binarizados

Agora em escala: trezentas e sessenta imagens sintéticas de oito por oito, com ruído de bit e sem rótulo nenhum. Reparem nos protótipos ao longo das iterações — começam como puro chuvisco e, depois de algumas rodadas, aparecem três protótipos nítidos. O algoritmo não fazia ideia de que existiam três dígitos ali, ele só maximizou verossimilhança.

E o protótipo final é suave, com tons de cinza: isso não é defeito, é informação — o cinza diz quais pixels são estáveis no grupo e quais variam.

Dois modos de quebrar o EM: verossimilhança nula e subfluxo numérico

Duas coisas quebram o EM na prática, e as duas são numéricas. A primeira: se um bit nunca acendeu no grupo k , a estimativa crava μ igual a zero exato. Aí chega uma amostra nova com aquele bit ligado, o produtório inteiro zera, o componente declara aquela amostra impossível e a normalização do passo E vira zero dividido por zero.

A segunda: mesmo sem zeros, eu multiplico centenas de números menores que um. O gráfico mostra o produto despencando até atravessar o menor float64 e virar zero absoluto. Duas doenças diferentes, e uma cura pra cada: suavização de Laplace pra primeira, espaço logarítmico pra segunda.

Solução 1: Suavização de Laplace e Priors Beta para evitar $\mu \rightarrow 0$

A primeira cura. Em vez de estimar μ pela razão pura das contagens, eu somo pseudo-contagens: alfa no numerador e dois alfa no denominador. Isso é o mesmo que pôr uma priori Beta sobre o μ e tomar a moda da posteriori: não é gambiarra, é estimação bayesiana.

O efeito é simples: com alfa maior que zero, o μ nunca encosta em zero nem em um. E a interpretação é humana: é como se eu já tivesse visto, antes dos dados, meia observação de cada lado. Com muitos dados o alfa some no meio das contagens; com poucos, é ele que impede uma conclusão absoluta a partir de evidência magra.

Solução 2: O truque numérico Log-Sum-Exp (LSE)

A segunda cura é o log-sum-exp, um truque que vale pra vida inteira. O problema: eu preciso do log da soma de exponenciais, e se os expoentes forem tipo menos oitocentos, cada exponencial vira zero na máquina e eu acabo com log de zero, menos infinito.

A solução é fatorar o maior deles. Coloco exp do $a_{\max}$ em evidência, aplico o log, e ele vira $a_{\max}$ mais o log da soma de exp de a_k menos $a_{\max}$. O maior expoente virou exp de zero, que é um, então a soma nunca some.

É matematicamente idêntico, e é a diferença entre o código rodar e cuspir NaN.

Laboratório: underflow ao vivo (float64 colapsando vs espaço logarítmico)

Aqui dá pra ver os dois lados na mesma tela. À esquerda, o cálculo ingênuo em float64: aumentando a dimensão, o produto cai — dez elevado a menos cem, menos duzentos, menos trezentos — e de repente crava zero e não sai mais de lá.

Morreu a informação. À direita, o mesmo cálculo em espaço logarítmico: em vez de multiplicar eu somo logaritmos, e a linha desce tranquila, sem degrau, porque menos setecentos é um número comum pra um float. Mesma matemática — só que uma implementação sobrevive e a outra não.

Código Python: mistura de Bernoulli com EM estável

E aqui está tudo junto em código. Reparem que o logsumexp está implementado com o truque do máximo e que o passo E inteiro acontece em log: eu somo log de μ e log de $1 - \mu$, nunca multiplico probabilidades. À direita, a saída real da execução.

Gerei três protótipos de seis bits, quarenta cópias de cada, com dez por cento dos bits invertidos por ruído. O modelo recuperou pesos de trinta e um, trinta e quatro e trinta e cinco por cento — quase o um terço verdadeiro. E dá pra ler o protótipo em cada linha da matriz de μ : perto de zero vírgula nove nos bits do padrão, perto de zero vírgula zero cinco nos outros.

E o log da verossimilhança subiu de menos quinhentos e vinte e nove pra menos trezentos e trinta e quatro, monótono, como a teoria prometia.

Síntese do Bloco 3 — do hipercubo binário ao espaço contínuo

Fechando o Bloco 3. O que fica é: no hipercubo binário a distribuição nativa é a Bernoulli multivariada, e o protótipo vira um vetor de probabilidades, não um ponto no espaço. O somatório preso no log impede solução fechada, e o EM contorna com duas atualizações analíticas que nunca pioram a verossimilhança.

E o agrupamento virou suave: cada amostra distribui crédito, em vez de ser carimbada num grupo só. E reparem que a estrutura E-M não tem nada de específico da Bernoulli: aquelas duas equações valem pra qualquer família, muda só o que entra no lugar de p de x dado θ .

Trocando a Bernoulli por uma gaussiana, eu ganho o modelo pra dados contínuos, com forma, orientação, elipse. E é isso que a Bianca vai apresentar. Bianca, é com você.

GMM, demonstração e comparação

Bianca Fernandes Visco

Slides 53–67 · 15 slides · 1954 palavras · $\approx$ 14:28 de fala

SLIDE 53 NÚCLEO

141 palavras · $\approx$ 1:03

GMM: combinação convexa de elipsóides gaussianos

Obrigada, Antonio. Eu sou a Bianca Visco e vou fechar a apresentação com o modelo mais geral dos três: o Gaussian Mixture Model. O Antonio mostrou que o EM não é um algoritmo da Bernoulli, é um esquema que serve pra qualquer família. Então a gente troca a peça de dentro: no lugar da Bernoulli entra a gaussiana multivariada, e o modelo vale pra dados contínuos.

A fórmula é uma combinação convexa: os pesos π_k são positivos e somam um, então o resultado continua sendo uma densidade legítima. E o ganho está no segundo parâmetro de cada componente. Antes eu tinha só o centro; agora eu tenho também a matriz de covariância Σ_k , que carrega a forma e a orientação daquele grupo.

Por isso na figura os componentes não são bolinhas: são elipses, cada uma com seu tamanho, alongamento e inclinação.

SLIDE 54 NÚCLEO

135 palavras · $\approx$ 1:00

A anatomia da Gaussiana Multivariada: expoente quadrático e normalização

Vamos abrir a gaussiana multivariada em duas partes. A primeira é o termo de normalização, esse Z embaixo da fração. O determinante de Σ mede o hipervolume do elipsóide, e ele está ali garantindo que a integral da densidade dê exatamente um. Na prática: se o grupo é muito espalhado, o pico tem que ser mais baixo.

A segunda parte é o expoente, que é uma forma quadrática — e essa forma quadrática é a distância de Mahalanobis ao quadrado. Reparem no que ela substituiu: no K-Means a distância era x menos μ transposto vezes x menos μ , e aqui entrou a inversa de Σ no meio.

Esse sanduíche é a diferença entre medir distância com uma régua igual em todas as direções e medir com uma régua que se adapta à forma da nuvem.

A Matriz de Covariância 2×2 desmistificada: variâncias e correlação ρ

Vale destrinchar a matriz dois por dois, que é como a gente entende e depois generaliza. Na diagonal estão as variâncias de cada eixo: sigma um ao quadrado é o quanto a nuvem se espalha na horizontal, sigma dois ao quadrado na vertical. Fora da diagonal está a covariância cruzada, que dá pra reescrever como ρ vezes sigma um vezes sigma dois, onde ρ é a correlação, entre menos um e mais um.

Duas leituras. Se ρ é zero, a matriz fica diagonal e a elipse tem os eixos alinhados com os do gráfico. Se ρ é diferente de zero, a elipse inclina: positivo pra direita, negativo pra esquerda. E a matriz precisa ser simétrica e positiva-definida: se não for, não descreve elipse nenhuma e a densidade nem existe.

Laboratório da covariância: o que a matriz Σ realmente faz

Aqui dá pra sentir com a mão. Tenho três controles: as duas variâncias e a correlação. Aumento sigma um e a elipse estica na horizontal; aumento sigma dois e ela estica na vertical. Com os dois iguais e ρ zero eu tenho o círculo, que é o caso do K-Means.

Agora eu levo o ρ pra mais zero vírgula oito e olhem — a elipse gira e fica inclinada, contando que quando x um é alto, x dois também tende a ser. Puxo pro negativo e ela inclina pro outro lado. Repare também nos eixos desenhados dentro: são os autovetores da matriz, com comprimento igual à raiz do autovalor.

É essa a decomposição espectral: a covariância é uma rotação seguida de um esticamento.

Distância de Mahalanobis: a métrica que enxerga a geometria dos dados

Agora, por que a Mahalanobis é a métrica certa. Imaginem uma nuvem alongada na diagonal e dois pontos à mesma distância euclidiana do centro: um na direção do alongamento, outro na direção estreita. Pra régua euclidiana, os dois são igualmente típicos. Mas não são: o que está na direção estreita é muito mais estranho, porque naquela direção a nuvem quase não varia.

A Mahalanobis conserta isso medindo em unidades de desvio padrão local: divide cada direção pela dispersão que os dados realmente têm ali. E o detalhe elegante é o caso particular: se sigma for a matriz identidade, a inversa também é a identidade, o sanduíche colapsa e a Mahalanobis vira exatamente a distância euclidiana.

Ou seja, o K-Means é o caso pobre desse modelo.

Laboratório: Mahalanobis vs Euclidiana (a métrica que enxerga correlações)

E aqui está o argumento na tela, com uma nuvem inclinada e dois pontos de teste. Olhem os círculos tracejados: são os contornos da euclidiana, sempre redondos, indiferentes à forma dos dados. Agora ligo os contornos de Mahalanobis, e eles são elipses que acompanham a nuvem.

Vejam esses dois pontos: pela euclidiana, os dois estão à mesma distância. Pela Mahalanobis, esse aqui está a uma unidade e meia, e esse outro a quase quatro. O segundo é o ponto realmente anômalo, e só a Mahalanobis percebe. Guardem essa ideia, porque daqui a pouco ela vira detecção de anomalia.

EM para GMM: agora o Passo M também ajusta a forma

O EM para GMM é o mesmo esqueleto do Bloco 3, com uma equação a mais. O passo E é idêntico: a responsabilidade é o peso vezes a densidade gaussiana, normalizado pela soma. No passo M, o π_k continua sendo N_k sobre N e o μ_k continua sendo a média ponderada pelas responsabilidades — igualzinho à Bernoulli.

A novidade é a terceira equação: o σ_k é a matriz de covariância das amostras em torno do novo centro, também ponderada pelas responsabilidades. É aí que mora a diferença pro K-Means, que atualiza só a posição: o GMM atualiza posição, tamanho, alongamento e ângulo.

É por isso que, na figura, as elipses não só andam, elas também giram e mudam de proporção ao longo das iterações.

Laboratório: animação interativa da convergência do EM-GMM

Rodando ao vivo. No começo as elipses estão todas parecidas, mais ou menos circulares, porque foram inicializadas assim. A cada passo E as cores dos pontos ficam misturadas, degradê, porque um ponto na região de sobreposição realmente pertence um pouco a cada componente. E a cada passo M as elipses se ajustam.

Vejam essa aqui — ela vai esticando e girando até se encaixar na faixa diagonal. E repare que os pontos na fronteira entre dois grupos continuam com cor misturada até o fim. Isso não é o modelo falhando, é o modelo sendo honesto: ali existe ambiguidade de verdade, e o GMM guarda essa informação em vez de jogar fora.

Singularidades da Gaussiana e a regularização `reg_covar`

Mas essa flexibilidade toda tem um preço, e é uma patologia séria. Se um componente colapsar em cima de um único ponto, o sigma dele vai a zero, o determinante vai a zero, e como o determinante está no denominador, a densidade naquele ponto vai pro infinito.

A verossimilhança explode. E o pior é que isso é um máximo legítimo — o EM está fazendo o trabalho dele, subindo a verossimilhança, e mesmo assim o resultado é lixo: um componente que descreve um ponto só. Puxando o controle aqui, vocês veem a elipse encolhendo e o log da verossimilhança disparando.

A solução prática é um piso: somar ϵ vezes a identidade na diagonal, o que no scikit-learn se chama `reg_covar` e vem com dez elevado a menos seis por padrão. Isso impede o determinante de chegar a zero e mata a singularidade.

As quatro geometrias de `covariance_type` — e o preço de cada uma

O scikit-learn deixa escolher quanta liberdade geométrica dar, em quatro opções. Spherical: todo componente é um círculo, praticamente um K-Means probabilístico. Diag: elipses, mas com os eixos obrigatoriamente alinhados aos eixos do gráfico, sem inclinação. Tied: todos os componentes têm exatamente a mesma matriz, ou seja, a mesma forma e a mesma inclinação, mudando só o centro.

E full: cada componente com a sua matriz, liberdade total. Aí entra o critério de informação. Na tabela, o full tem mesmo a maior verossimilhança, menos mil trezentos e cinquenta e seis, mas gasta dezessete parâmetros. O tied tem verossimilhança quase igual com onze parâmetros, e por isso ele ganha, com BIC dois mil setecentos e oitenta e oito, que é o menor da coluna — é essa a célula destacada.

Faz sentido: esses dados foram gerados com elipses de mesma forma, então o tied é exatamente o modelo certo, e pagar por seis parâmetros a mais não compensa.

Laboratório: varredura de K e tipos de covariância com curva de BIC

Aqui dá pra varrer sozinho: eu escolho o tipo de covariância e o K, e a curva de BIC se redesenha. Reparem que ela tem um mínimo bem definido, diferente da inércia do K-Means, que só caía. É essa a vantagem do BIC: o termo de penalidade, p vezes $\log$ de N , cresce com o número de parâmetros, então em algum ponto ele passa a dominar o ganho de ajuste.

Aumentando o K além do verdadeiro, olhem: o BIC sobe. O modelo está dizendo, com número, que eu estou complicando à toa. E comparando os tipos: os mais flexíveis ganham em verossimilhança e perdem em penalidade, e quem vence depende de como os dados realmente são.

Demonstração: K-Means e GMM sobre o mesmo conjunto anisotrópico

Agora a demonstração que amarra tudo: mesmo conjunto, três faixas alongadas e inclinadas, dois algoritmos. À esquerda o K-Means com n_init igual a dez, ou seja, dez sementeiras ficando com a melhor: ele corta as faixas transversalmente e atinge sessenta e três por cento de pureza.

E não adianta rodar mais vezes: o problema não é inicialização, é a hipótese — fronteiras de Voronoi são retas, e faixas inclinadas não se separam assim. À direita, o GMM com `covariance_type full`, e o resultado é cem por cento. As elipses simplesmente abraçam cada faixa.

E reparem no código: são quatro linhas de scikit-learn, praticamente a mesma interface. O que mudou não foi a dificuldade de implementar, foi a hipótese sobre a forma dos grupos.

Laboratório: detecção de anomalia por limiar de densidade $p(x) < \tau$

E tem um bônus que só o GMM entrega, por ser generativo: detecção de anomalia. Como eu tenho a densidade p de x em todo ponto do plano, posso fixar um limiar τ e declarar anomalia tudo que cair abaixo dele. Arrastando a amostra de teste pra dentro da nuvem: densidade alta, padrão normal.

Levo pra região vazia entre os grupos e a densidade despenca, disparando o alerta. E agora o limiar. Repare na área sombreada e na curva que marca p de x igual a τ : quando eu aumento o $\ln$ de τ , a fronteira se abre e o modelo fica mais rigoroso, condenando mais região do plano como anômala; quando eu diminuo, a fronteira se fecha e ele fica mais permissivo.

É exatamente o trade-off entre falso positivo e falso negativo, e ele fica visível aqui.

Quadro comparativo dos três métodos

Esse quadro resume os três métodos lado a lado. Domínio: K-Means e GMM em espaço contínuo, Bernoulli no hipercubo binário. Hipótese: hiperesferas, Bernoullis independentes, elipsóides. Atribuição: rígida no K-Means, suave nos outros dois. Critério: o K-Means minimiza uma inércia, os modelos de mistura maximizam uma verossimilhança.

E a linha mais importante é a da capacidade generativa. O K-Means não tem: ele particiona, mas não sabe gerar um dado novo nem dizer o quanto um dado é provável. Os modelos de mistura têm, e é daí que saem coisas como detecção de anomalia, imputação de valores faltantes e amostragem sintética.

A regra prática é simples: se os grupos são redondos, parecidos e você quer velocidade, K-Means resolve; se tem forma, sobreposição ou você precisa de incerteza, vá de mistura.

Conclusões, referências e abertura para perguntas

Pra fechar, três mensagens. Primeira: é tudo um arcabouço só — a variável latente discreta está por trás dos três métodos, muda só o que a gente assume sobre como os dados nascem. Segunda: o K-Means é rápido e continua ótimo pra muita coisa, mas descarta a incerteza e impõe esferas, e isso cobra um preço quando os dados têm forma.

Terceira: o EM dá modelos que descrevem a incerteza e ainda são generativos. E olhem a fórmula do elo formal, que é o fecho mais bonito do trabalho: se você fixa todas as covariâncias em ϵ vezes a identidade, iguala os pesos e faz ϵ tender a zero, as responsabilidades suaves colapsam em zero e um, e o EM vira, literalmente, o algoritmo de Lloyd.

Ou seja, o K-Means não é um método diferente: é o caso-limite determinístico do GMM. As referências principais são o Bishop, capítulo nove, e a documentação do scikit-learn. Era isso. Muito obrigada pela atenção, e estamos abertos às perguntas.